Information Sciences 546 (2021) 25–41

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Scalable and redactable blockchain with update and anonymity

Ke Huang [a,*], Xiaosong Zhang [a,b], Yi Mu [c], Fatemeh Rezaeibagha [d], Xiaojiang Du [e]

[a] Research College for Cyber Security, University of Electronic Science and Technology of China (UESTC), Chengdu 611731, China
[b] Cyberspace Security Research Center, Peng Cheng Laboratory, Shenzhen 518055, China
[c] Key Laboratory of Network Security and Cryptology, College of Mathematics and Informatics, Fujian Normal University, Fuzhou 350007, China
[d] Information Technology, Mathematics and Statistics Discipline, Murdoch University, Perth 6150, Australia
[e] Department of Computer and Information Sciences, Temple University, Philadelphia, PA 19122, USA

ARTICLE INFO

Article history:
Received 25 September 2019
Received in revised form 6 July 2020
Accepted 8 July 2020
Available online 9 August 2020

Keywords:
Blockchain
Internet-of-Things
Redaction
Spontaneous Ring

ABSTRACT

Internet-of-Things (IoT) envisions communications between heterogeneous devices and utilization of the associated data for smart decision making. Blockchain bridges the gap between widely-distributed IoT devices and the need for a universal trust-layer. When applying blockchain for IoT, some concerns can arise. Among them, scalability is a crucial factor because it decides whether blockchain can keep empowering IoT in the long term. According to a recent survey by Ali et al., some newly launched blockchains are suffering from powerful attacks (e.g. 51% attack) when the computing pool is small, and the number of participated nodes is inadequate (USENIX 2016). To ensure the scalability of initially-deployed blockchains, a proper countermeasure is important to be devised. Ateniese et al. proposed the notion of the redactable blockchain (EuroS&P 2017) which allows block history to be rewritten by using chameleon hash. However, the distribution and management of trapdoor key is crucial for the scalability of chameleon hash as well as redactable blockchain. To deal with the above problems, we propose two cryptographic schemes as the new theoretic tools for blockchain redaction: time updatable chameleon hash (TUCH) and linkable-and-redactable ring signature (LRRS). The use of TUCH and LRRS schemes enables redaction to take place scalably and anonymously where the spontaneous ring is generated for redaction, which costs little expenditures to rewrite a block content. Specifically, the redaction will be processed without assigning and splitting trapdoor key among multiple users in a complex way, and achieve transaction anonymity for users. What is more, we briefly instantiate how to build a redactable blockchain with update and anonymity (SRB) with our proposed TUCH and LRRS. While security analysis confirms that our proposals are theoretically secure, the experimental results show that our proposals are efficient for implementation purposes.

1. Introduction

The advancements of societies are driven by intelligent utilization and management of data generated by heterogeneous devices, such as sensor-enabled types of machineries [24]. How to accommodate, analyze and benefit from massive heterogeneous data is the most crucial factor for realizing the Internet-of-Things (IoT) [4]. The rise of blockchain [36] offers an ideal

* Corresponding author.
E-mail addresses: huangke@uestc.edu.cn (K. Huang), johnsonzxs@uestc.edu.cn (X. Zhang), ymu.ieee@gmail.com (Y. Mu), fatemeh.rezaeibagha@murdoch.edu.au (F. Rezaeibagha), dxj@ieee.org (X. Du).

https://doi.org/10.1016/j.ins.2020.07.016
0020-0255/

solution to this problem by establishing a trust-free public layer among IoT devices without a central party. To implement scalable and secure blockchain systems for industrial activities, relevant issues have been extensively discussed [32]. Among them, scalability is a vital subject to study as it is necessary for the adoption of blockchain in the foreseeable future. Scalability usually implies scalable consensus mechanism [43] and basic primitives (such as: hash function and digital signature) used to power blockchain [21]. In this work, we focus on the scalable and secure hashing and signing schemes for blockchain.

The increasingly powerful attacks against the blockchain are pushing it to the age of transformation. According to a recent report by Ali et al. [8], some newly launched blockchains are vulnerable to powerful attacks (such as 51% attack [36]) due to inadequate participated nodes and small computing pool. The evidence showed that some chains are manipulated by a large pool which controls over 60% of the network power. Consequently, some illegal contents [7] and unwanted information [35] were forced to be recorded on blockchain, and they are hard to remove or reverse due to immutable feature of blockchain. A straightforward resolution has been proposed by Ateniese et al. [7] to re-write block history back to a normal state once the chain is compromised. Their core principle is to replace the hash used in blockchain with a chameleon hash function [28] which creates a backdoor to the hash block. Simply, it allows to find a different nonce or random number for a given block without changing its hash value. Therefore, block contents can be written without causing inconsistency to block hash.

Basically, studies of redactable blockchain (or mutable blockchain [20]) seek to break or bypass blockchain's immutability to compensate for fatal errors found in distributed ledger. Another strong motivation is erasing individual privacy or unwanted information from blockchain to fulfil "the right to be forgotten" demanded by GDPR's regulation [40]. In this work, we put aside the regulation issues (top-level design and legal concerns) and mainly focus on the technical design and implementation of the core cryptographic protocols for redaction. As we later survey in Section 2.3, some current works (e.g., [9]) based on chameleon hash result in evident costs to distribute trapdoor key among multiple nominated parties for collaboration. Meanwhile, as we revealed in our previous work [26], the cost of forming a non-spontaneous ring for each member who holds a piece of a trapdoor key is evidently high and becomes even higher with the number of nodes. Meanwhile, it also causes misuses of redaction power. Suppose anyone who derives hash collision elsewhere by either brutal-force computing or eavesdropping others, he can redact the chain whenever he is willing to. The above drawbacks indicate the scalability problem of redactable blockchain. If we do not solve it, redactable blockchain can not be considered as an ideal trust-layer for IoT network or any other platforms in the long run.

In this paper, we propose a time updatable chameleon hash (TUCH) and linkable-and-redactable ring signature (LRRS) as theoretical building blocks to construct a redactable blockchain with update and anonymity (SRB). The improvements to our previous work [26] is that the hashing scheme TUCH we proposed is based on a spontaneous ring which is scalable and efficient to deploy and use. Meanwhile, a linkable and redactable ring signature is devised to capture the anonymity and double-spending issues of blockchain. Above proposals help build a scalable and redactable blockchain (SRB). Our contributions can be summarized as the following:

1) We propose the first time updatable chameleon hash (TUCH), which allows for spontaneous ring formation and does not rely on the interactive participation of any other entities for the key generation. The chameleon randomness for holding a hash collision is under periodic expiration, it can pass verification only within a fixed time window (i.e., an hour or a day, represented by Δt).
2) We propose the first linkable-and-redactable ring signature (LRRS), which allows a signer to sign a message anonymously without relying on any trusted entities. A ring of users is formed spontaneously where the members are unaware that their public keys are selected for creating the signature. Meanwhile, the LRRS signature can be redacted to commit to another message by the designated holder of the trapdoor key. Also, LRRS offers linkability, which links two anonymous signatures generated by the same signer; this is useful to detect double-spending in crypto-currencies.
3) We instantiate how to apply our proposed TUCH and LRRS in public blockchain for redaction. We also give security analysis based on pre-defined security models to achieve security requirements. The analysis shows that our proposals are secure against defined adversarial models. We also perform experiments to evaluate the performance of our proposed schemes. Simulation results show that our proposed TUCH is scalable and efficient in practice. Meanwhile, LRRS is less efficient than peer works but at an acceptable level. It traded efficiency for anonymity, and this deficiency is factored by the size of the selected ring.

The remainder of this paper is organized as follows. In Section 2, we present the background for this work; In Section 3, we provide system model and definitions; In Section 4, we present constructions and security analysis of our proposed TUCH and LRRS schemes; In Section 5, we show instantiation of applying TUCH and LRRS to build SRB; In Section 6, we provide performance evaluations; Section 7 concludes the paper.

2. Related work

In this section, we review some related works of chameleon hash, ring signature and redactable blockchain as follows.

2.1. Chameleon hash

Chameleon hash is a trapdoor one-way hash function where finding a chameleon hash is hard without the trapdoor key. The first idea was proposed by Krawczyk et al. [28] where the notion of chameleon signature (CS) was also revealed. CS is a simpler and non-interactive version of undeniable signatures [19]. In an undeniable signature, one has to interact with the signer to verify the validity of a signature (via a deniable protocol). Later on, the key-exposure problem is identified by Ateniese et al. [2], which captures the leakage of trapdoor key in any given hash collisions. Since then, several schemes with key-exposure freeness were proposed, such as [18,13,27].

Camenisch et al. [13] proposed an ephemeral chameleon hash which prevents forging a collision without the ephemeral trapdoor. They formalized definitions and security for chameleon hash function in [13]. To clarify, we review some related works in Table 1. As it is shown in Table 1, chameleon hash schemes are compared by involvements of ID-based infrastructure, pairing computations, collision-resistance and intractability assumptions. As it is shown in Table 1, the majority of the listed schemes do not rely on ID-based infrastructure and pairing computation.

2.2. Ring signature

Ring signature was originated from group signature [16]. In group signature, to generate a signature, a group of users are summoned by a group manager to form the group. Any user in this group can sign on behalf of the whole group. As a result, anonymous authentication is achieved. Unlike the group signature, the formation of the ring signature is spontaneous where the ring members are unaware of the fact that they joined the ring. There are different types of ring signatures including threshold version [12], revocable version [5] and linkable version [42]. Due to development of crypto-currencies, linkable ring signature is gaining massive attention, as it preserves user's identity, meanwhile detecting any doubly used crypto-currencies [29]. However, despite the explored security and efficiency features to enhance linkable ring signature, a redactable version of any group or ring-based signatures are yet to study. We review some relevant works in Table 2. As it is shown in Table 2, most ring signature schemes are based on public-key infrastructure, and are linkable. In addition, few works achieved the design of constant signature size (i.e., $|\sigma| = \mathcal{O}(1)$).

Table 1
Overview of chameleon hash works.

Scheme	Identity-based	Pairing-based	Collision-resistance	Assumptions
ICH [2]	Yes	No	Yes	q-SDHP
KEF [18]	No	No	Yes	CDHP
CHE [13]	No	No	Yes	RSA and DLP
CHD [27]	No	No	Yes	RSA
ACH [31]	No	Yes	Yes	CDHP
CHW [17]	No	No	Yes	CDHP
CIK [15]	No	No	Yes	Factoring
ACC [11]	No	No	Yes	Factoring

Each work is named after the abbreviation of article title.

Table 2
Overview of ring signature works.

Scheme	Crypto-system	Pairing-based	Signature size	Linkability	Assumptions
SLT [42]	PKI	No	$\mathcal{O}(n)$	Yes	DDHP, S-RSA
SLRSE [41]	PKI	No	$\mathcal{O}(1)$	Yes	S-RSA
SLRSR [1]	PKI	No	$\mathcal{O}(n)$	Yes	DDHP, S-RSA
LRSS [33]	PKI	No	$\mathcal{O}(n)$	Yes	DDHP
RCT2.0 [39]	PKI	Yes	$\mathcal{O}(1)$	Yes	DDH, k-SDH
LRSU [30]	PKI	No	$\mathcal{O}(n)$	Yes	DLP
ELT [45]	PKI	Yes	$\mathcal{O}(\sqrt{n})$	Yes	SDH, DDHI
SIB [6]	IDB	Yes	$\mathcal{O}(1)$	Yes	DLP, DDHP
ESL [23]	PKI	Yes	$\mathcal{O}(\sqrt{n})$	Yes	SDA, q-SDH
TRS [22]	PKI	No	$\mathcal{O}(n)$	No	DDHP

Denote k-SDH as k-Strong Diffie-Hellman Assumption [39]; q-SDH as q-Strong Diffie-Hellman Assumption; DDHI as q-Decisional Diffie-Hellman Inversion Assumption; SDA as Subgroup Decision Assumption; S-RSA as Strong RSA Assumption; PKI as Public-Key based Infrastructure; IDB as Identity-based Infrastructure. Each work is named after the abbreviation of the article title.

Table 3
Overview of redactable blockchain works.

Scheme	Technique & Principle	Pros & Cons
Reversecoin [38]	uses offline key to reverse transaction by a configurable timeout period	the first revocable coin, but does not succeed
Redactable Blockchain [7]	proposes the notion of redactable blockchain formally with chameleon hash	allows to rewrite block history, but many issues are not discussed
Thwarting Insertion [35]	proposes a content detector with minimum fees to reject malicious contents on chain	gives conceptual suggestions, but does not provide formal security analysis
μchain [37]	proposes μchain to enable changes of transaction state to erase relevant blockchain records	does not rely on complex cryptographic schemes, but introduces high computing costs
Permissionless [20]	proposes voting-based consensus protocol to achieve redactable blockchain	relies on complex cryptographic protocol, but it is fairly efficient to run
Enhanced Redactable Blockchain [9]	uses multi-party computation to reconstruct the trapdoor for secure redaction among multiple validators	enhances redaction security at the expense of introducing complex interactions and computings

Other related works are [26,25], each scheme is designed for specific application scenario.

2.3. Redactable blockchain

Redactable blockchain refers to a blockchain with editable (redactable) block or revocable transaction. The fundamental motivation of this primitive is to recover distributed ledger from fatal error without causing hash inconsistency or any hard forks. As shown in Table 3, various schemes were proposed to achieve this goal.

Reversecoin is the first crypto-currency to support transaction revocation. It allows the user to reverse a transaction in a configurable timeout by applying off-line keys. Although this project does not succeed, it provides valuable insight in designing redactable blockchain.

Ateniese et al. [7] proposed the notion of redactable blockchain in EuroS&P 2017. They introduced chameleon hash as the core element to achieve redactable blockchain so that block contents could be re-written efficiently. However, there are still some problems left to be answered for. For example, how to reach an agreement on redaction or how chameleon hash is programmed to achieve redaction. Then, Matzutt et al. [35] proposed a content detector with minimum fees to reject malicious contents insertion on chain, but no security analysis is provided to validate their proposal. In addition, Puddu et al. [37] proposed μchain to alter transaction state in an effort to change blockchain records. Although no cryptographic solution is involved, their proposal induced high computing costs. Deuber et al. [20] proposed a consensus-based voting and policy to dictate redaction. Only a tiny overhead is yielded in their proposal in comparison with other works. Recently, Ashritha et al. [9] proposed to split trapdoor key (private key) among multiple validators and reconstruct it via multi-party computation for secure redaction. However, their solution relies on interactive protocol and is constrained by the number of participants.

Politou et al. [20] proposed a review article to list all notable challenges and developments in redactable blockchain. As discussed in their work, solutions for redactable blockchain were not limited by cryptographic methods. Possible solutions extended to consensus protocol, malware detection, adjustment of transaction fees, etc. The philosophy of the above solutions is to lead majority of nodes forget about the problematic block history and switch to the "right" chain. In a recent work, Xu et al. [44] proposed a blockchain-based identity management and authentication scheme. They adopted chameleon hash in order to delete illegal users' information without causing hash inconsistency.

3. System model and definitions

In this section, we first give preliminaries used in this work. Then, we show the system model for our scalable and redactable blockchain with update and anonymity (SRB). After that, we give definitions for our TUCH and LRRS schemes. They both serve as core elements to achieve SRB.

3.1. Preliminaries

Let G be a cyclic multiplicative group of prime order q and generator g. We have the following complexity assumptions in G:

- Discrete Logarithm Problem (DLP): Given $g^a \in G$ where $a \in_R Z_q$, computing a is hard.
- Computational Diffie-Hellman Problem (CDHP): Given $g, g^a, g^b \in G$ where $a, b \in_R Z_q$, computing g^{ab} is hard.
- Decisional Diffie-Hellman Problem (DDHP): Given $g, g^a, g^b, g^c \in G$ where $a, b, c \in_R Z_q$, deciding $c = ab$ is hard.
- A Gap Diffie-Hellman (GDH) group is the group where CDHP is hard and DDHP is easy. $< g, g^a, g^b, g^c >$ is a Diffie-Hellman tuple if and only if $c = abmodq$.

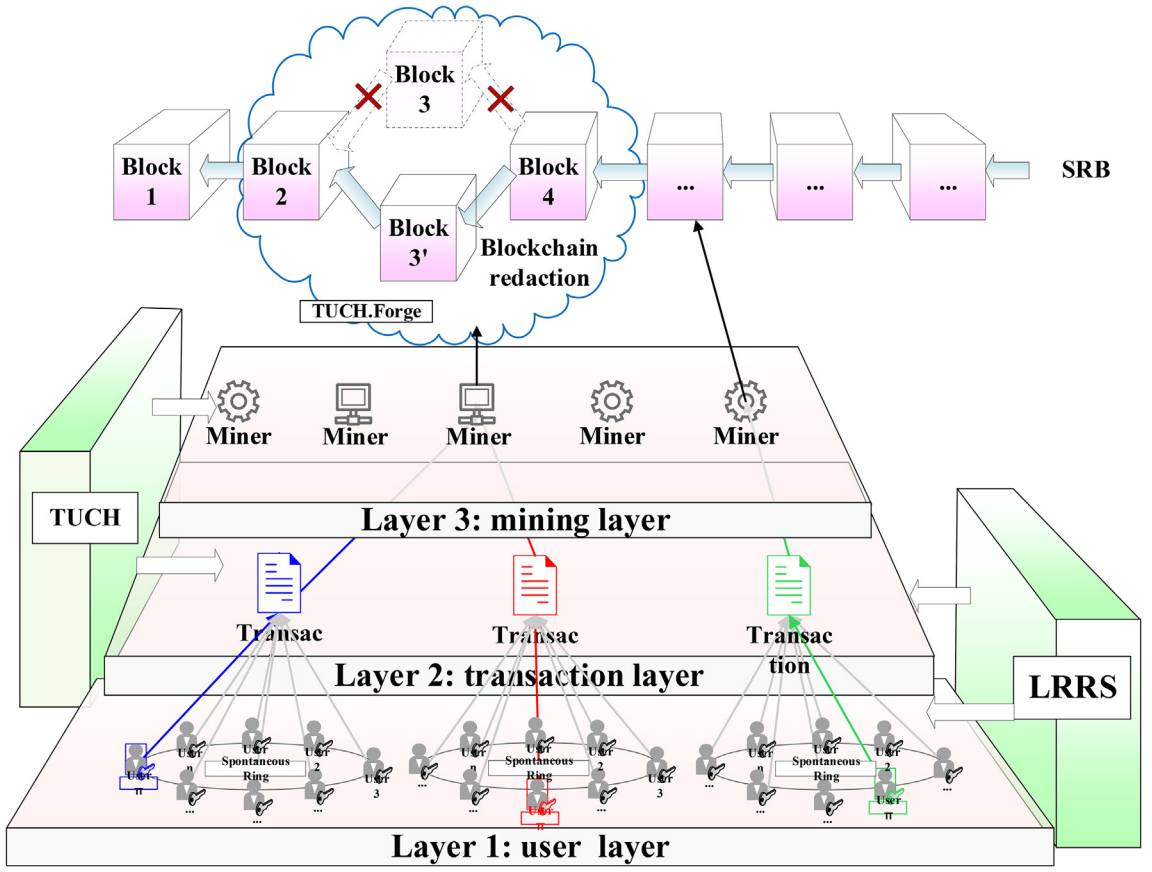

Fig. 1. The framework of scalable and redactable blockchain.

3.2. System model of SRB

The framework of our scalable and redactable blockchain with update and anonymity (SRB) is shown in Fig. 1. As coincided with public blockchain (i.e. bitcoin [36]), our SRB includes two main parties defined as below.

Miner (or trapdoor holder): Entities which pack signed transactions from users and record them in blockchain based on the proof-of-work consensus [36]. Miner runs TUCH.KeyGen to generate trapdoor key tk and hash key hk. Hash key hk is a public key used to generate a chameleon hash $\hbar$. Trapdoor key tk is a private key kept by the miner secretly, it is used to produce a hash collision for chameleon hash h to which corresponding hash key hk is committed.

User: Entities sign and publish transaction in order to record it on the chain. For a user i, he runs LRRS.UKeyGen to generate signing key sk_i and verification key pk_i. He keeps sk_i privately and runs LRRS.Sign to produce a LRRS signature for a transaction.

As it is shown in Fig. 1, our SRB is designed to be scalable and decentralized that no central authority is established. The workflow of the blockchain insertion is similar to bitcoin [36] where the user publishes signed transactions, and miners verify and record them in blockchain. By replacing ordinary hash function (e.g. SHA-256) and elliptic curve digital signing algorithm (ECDSA) used in standard blockchain with our TUCH and LRRS, we can transform ordinary chain to a redactable chain. An instantiation is given in Section 5.

3.3. Definitions of TUCH

Definition 1. Our Time Updatable Chameleon Hash (TUCH) consists of the following six algorithms:

- **TUCH.Setup**$(\lambda) \rightarrow (\mathsf{param}_{\mathsf{TUCH}})$: On input a security parameters λ, output system parameters $\mathsf{param}_{\mathsf{TUCH}}$.
- **TUCH.KeyGen**$(\mathsf{param}_{\mathsf{TUCH}}) \rightarrow (tk, hk)$: On input system parameters $\mathsf{param}_{\mathsf{TUCH}}$, output trapdoor key and hash key (tk, hk).

- **TUCH.Hash**$(hk, \mathsf{CID}, m, t, \alpha) \rightarrow (\hbar, r)$: On input a hash key hk, a customized identity CID, a message m, a current time t and a randomness α, output chameleon hash and chameleon randomness $(\hbar, r)$.
- **TUCH.Verify**$(hk, \mathsf{CID}, (m, \hbar, r), t) \rightarrow (\perp, 0 \text{ or } 1)$: On input a hash key hk, a customized identity CID, a tuple $(m, \hbar, r)$ which consists of message m, chameleon hash $\hbar$ and chameleon randomness r, and a current time t, output $\perp$, 0 or 1.
- **TUCH.Forge**$(tk, \mathsf{CID}, m', (m, \hbar, r), t) \rightarrow (r' \text{ or } \perp)$: On input a trapdoor key tk, a customized identity CID, a new message m', a tuple $(m, \hbar, r)$ and a current time t, output a new chameleon randomness r' or $\perp$.
- **TUCH.Update**$(tk, \mathsf{CID}, (m, \hbar, r, t), \Delta t) \rightarrow (r'' \text{ or } \perp)$: On input a trapdoor key tk, a customized identity CID, a tuple $(m, \hbar, r, t)$, and a time interval Δt, output an updated chameleon randomness r'' or $\perp$.

3.4. Definitions of LRRS

Definition 2. Our Linkable-and-Redactable Ring Signature (LRRS) consists of the following nine algorithms:

- **LRRS.Setup**$(\lambda) \rightarrow (\mathsf{param}_{\mathsf{LRRS}})$: On input a security parameter λ, output system parameters $\mathsf{param}_{\mathsf{LRRS}}$.
- **LRRS.UKeyGen**$(\mathsf{param}_{\mathsf{LRRS}}) \rightarrow (sk_i, pk_i)$: On input system parameters $\mathsf{param}_{\mathsf{LRRS}}$, output user's private and public key pair (sk_i, pk_i).
- **LRRS.HKeyGen**$(\mathsf{param}_{\mathsf{LRRS}}) \rightarrow (hk, tk)$: On input system parameters $\mathsf{param}_{\mathsf{LRRS}}$, output trapdoor key tk and hash key hk.
- **LRRS.Sign**$(hk, L, m, x_\pi, t) \rightarrow (\sigma_L^t(m))$: On input a hash key hk, a list of n public keys L, a message m to be signed, signer's private key x_π where $\pi \in [1, |L|]$ is confidential and hidden, and a current time t, output a LRRS signature at time t as $\sigma_L^t(m)$.
- **LRRS.Verify**$(L, m, \sigma_L^t(m), \bar{t}) \rightarrow (0 \text{ or } 1)$: On input a list of n public keys L, a message m, a LRRS signature at time t as $\sigma_L^t(m)$ and a current time $\bar{t}$, output 0 or 1.
- **LRRS.Redact**$(tk, L, m', \sigma_L^t(m), t) \rightarrow (\perp \text{ or } \sigma_L^t(m'))$: On input a trapdoor key tk, a list of n public keys L, a new message m', a LRRS signature at time t as $\sigma_L^t(m)$ and a time point t, output $\perp$ or a redacted LRRS signature $\sigma_L^t(m')$ at time point t.
- **LRRS.Update**$(tk, L, m, \sigma_L^t(m), t, \Delta t) \rightarrow (\perp \text{ or } \sigma_L^{t+\Delta t}(m))$: On input a trapdoor key tk, a list of n public keys L, a message m, a LRRS signature $\sigma_L^t(m)$ at time t, a time point t and a global time interval Δt, output $\perp$ or an updated LRRS signature $\sigma_L^{t+\Delta t}(m)$ at time point $t + \Delta t$.
- **LRRS.Link**$(L, (pk_{d_0}, m_{d_0}, \sigma_L^t(m_{d_0})), (pk_{d_1}, m_{d_1}, \sigma_L^t(m_{d_1})), \bar{t}) \rightarrow (\perp, 0 \text{ or } 1)$: On input a list of n public keys $L = \{y_1, \cdots, y_n\}$, two tuples $(pk_{d_i}, m_{d_i}, \sigma_L^t(m_{d_i}))$ where $d_i \in L$ and $i = (0, 1)$, and a current time $\bar{t}$, output $\perp$, 0 or 1.
- **LRRS.Deny**$(L, m^*, \sigma_t^L(m^*), m, \sigma_t^L(m), \bar{t}) \rightarrow (\perp, 0 \text{or} 1)$: On input a list of n public keys $L = \{y_1, \cdots, y_n\}$, dispute message m^* and signature $\sigma_t^L(m^*)$, original message m and signature $\sigma_t^L(m)$, a current time $\bar{t}$, output $\perp$, 0 or 1.

3.5. Security requirements

In this part, we give security requirements for our TUCH and LRRS schemes.

3.5.1. Security Requirements of TUCH

Definition 3. A secure time updatable chameleon hash should satisfy the following properties [18].

Correctness: Our proposed TUCH should satisfy correctness defined as below. It describes that any correctly computed, redacted or updated chameleon hashes should pass verification successfully.

$$\forall\lambda, \forall \mathsf{param}_{\mathsf{TUCH}} \leftarrow \mathsf{TUCH.Setup}(\lambda), (tk, hk) \leftarrow \mathsf{TUCH.KeyGen}(\mathsf{param}_{\mathsf{TUCH}}),$$

$$\forall(\hbar, r) \leftarrow \mathsf{TUCH.Hash}(hk, \mathsf{CID}, m, t, \alpha), r' \leftarrow \mathsf{TUCH.Forge}(tk, \mathsf{CID}, m', (m, \hbar, r), t),$$

$$\forall(r'') \leftarrow \mathsf{TUCH.Update}(tk, \mathsf{CID}, (m, \hbar, r, t), \Delta t)$$

$$\mathsf{TUCH.Verify}(tk, \mathsf{CID}, (m, \hbar, r), t) = \mathsf{TUCH.Verify}(tk, \mathsf{CID}, (m', \hbar, r'), t) =,$$

$$\mathsf{TUCH.Verify}(tk, \mathsf{CID}, (m, \hbar, r''), t + \Delta t) = 1$$

Indistinguishability: Indistinguishability asks that no adversary can efficiently distinguish between the chameleon randomness generated from TUCH.Hash and TUCH.Forge. Our proposed TUCH is indistinguishable if for any efficient adversary $\mathscr{A}$ against our TUCH, we have probability $|\Pr[\mathsf{Ind}_{\mathscr{A}}^{\lambda} = 1] - \frac{1}{2}| \leqslant \nu(\lambda)$ in the experiment $\mathsf{Ind}_{\mathscr{A}}^{\lambda}$ defined as below where ν is a negligible function.

Experiment: $\mathsf{Ind}^{\lambda}_{\mathscr{A}}$

$\mathsf{param}^{\mathsf{ch}}_{\mathsf{TUCH}} \leftarrow \mathsf{TUCH.Setup}(\lambda)$

$(hk_{ch}, tk_{ch}) \leftarrow \mathsf{TUCH.KeyGen}(\mathsf{param}^{\mathsf{ch}}_{\mathsf{TUCH}})$

$a \xleftarrow{R} \{0,1\}$ where R indicates random selection

$b \xleftarrow{R} \mathscr{A}^{\mathsf{Hash\&Forge}(tk_{ch},\cdots,a),\mathsf{Forge}(tk_{ch},\cdots)}(hk_{ch})$

Here, oracle Hash&Forge on input

$(tk_{ch}, m, m', \mathsf{CID}, \bar{t}, a)$ where $\bar{t}$ is the current time

Set $(\hbar, r) \leftarrow \mathsf{TUCH.Hash}(hk_{ch}, \mathsf{CID}, m, \bar{t}, \alpha)$

Set $(\hbar', r') \leftarrow \mathsf{TUCH.Hash}(hk_{ch}, \mathsf{CID}, m', \bar{t}, \alpha')$

Set $(r'') \leftarrow \mathsf{TUCH.Forge}(tk_{ch}, \mathsf{CID}, m, (m', \hbar, r'), \bar{t})$

If $\mathsf{TUCH.Verify}(hk_{ch}, \mathsf{CID}, m', \hbar', r', \bar{t}) = \perp \ \vee r'' = \perp$

return $\perp$

If $a = 0$:

return $(\hbar, r)$

If $a = 1$:

return $(\hbar', r'')$

Return 1, if $b = a$;

else, return 0

Collision-resistance: Collision-resistance asks that even granted with access to a forging oracle Forge', no adversary can generate a fresh collision (never has been queried before) at the current time t while passing verification. Our proposed TUCH is collision resistant if for any efficient adversary $\mathscr{A}$ against our TUCH, we have the probability $\Pr[\mathsf{CollRes}^{\lambda}_{\mathscr{A}} = 1] \leqslant v(\lambda)$ in the experiment $\mathsf{CollRes}^{\lambda}_{\mathscr{A}}$ defined below where v is a negligible function.

Experiment: $\mathsf{CollRes}^{\lambda}_{\mathscr{A}}$.

$\mathsf{param}^{\mathsf{ch}}_{\mathsf{TUCH}} \leftarrow \mathsf{TUCH.Setup}(\lambda)$

$(hk_{ch}, tk_{ch}) \leftarrow \mathsf{TUCH.KeyGen}(\mathsf{param}^{\mathsf{ch}}_{\mathsf{TUCH}})$

$(\mathsf{CID}^*, m^*, r^*, m^{**}, r^{**}, \hbar^*, t^*) \leftarrow \mathscr{A}^{\mathsf{Forge}'(tk_{ch},\cdots),\mathit{Update}(tk_{ch},\cdots)}$

(hk_{ch})

Here, oracle Forge' on input $(tk_{ch}, m, m', \mathsf{CID}, t)$:

Return $\perp$ if $\mathsf{TUCH.Verify}(hk_{ch}, \mathsf{CID}, m, \hbar, r, t) \neq 1$

$r' \leftarrow \mathsf{TUCH.Forge}(tk_{ch}, \mathsf{CID}, m', (m, \hbar, r), t)$

If $r' = \perp$, return $\perp$; otherwise, return r'

Oracle Update on input $(tk_{ch}, \mathsf{CID}, (m, \hbar, r), t), \Delta t)$:

Return $\perp$ if $\mathsf{TUCH.Verify}\ (hk_{ch}, \mathsf{CID}, m, \hbar, r, t) \neq 1$

$r'' \leftarrow \mathsf{TUCH.Update}(tk_{ch}, \mathsf{CID}, (m, \hbar, r), \Delta t)$

Return 1 if $\mathsf{TUCH.Verify}(hk_{ch}, \mathsf{CID}^*, m^*, \hbar^*, r^*, \bar{t})$=

$\mathsf{TUCH.Verify}(hk_{ch}, \mathsf{CID}^*, m^{**}, \hbar^*, r^{**}, \bar{t}) = 1 \ \Lambda \ m^* \neq m^{**}$

else, return 0.

3.5.2. Security requirements of LRRS

Definition 4. A secure linkable-and-redactable ring signature should satisfy the following properties.

Correctness: Our proposed LRRS should satisfy correctness. It describes that any correctly generated, redacted or updated signatures should pass verification successfully.

$$\forall\lambda, \forall \mathsf{param}_{\mathsf{LRRS}} \leftarrow \mathsf{LRRS.Setup}(\lambda),$$

$$\forall n, L = \{pk_1, \cdots, pk_n\} \text{ where } pk_i \leftarrow \mathsf{LRRS.UKeyGen}(\mathsf{param}_{\mathsf{LRRS}}) \text{ for } (1 \leqslant i \leqslant n),$$

$$\forall (tk, hk) \leftarrow \mathsf{LRRS.HKeyGen}(\mathsf{param}_{\mathsf{LRRS}}), (\sigma^t_L(m)) \leftarrow \mathsf{LRRS.Sign}(hk, L, m, x_\pi, t),$$

$(\sigma_L^t(m')) \leftarrow \mathsf{LRRS.Redact}(tk, L, m', \sigma_L^t(m), t)$,

$\sigma_L^{t+\Delta t}(m) \leftarrow \mathsf{LRRS.Update}(tk, L, m, \sigma_L^t(m), t, \Delta t)$,

$\mathsf{LRRS.Verify}(L, m, \sigma_L^t(m), t) = \mathsf{LRRS.Verify}(L, m', \sigma_L^t(m')), t) =$

$\mathsf{LRRS.Verify}(L, m, \sigma_L^{t+\Delta t}(m)), t + \Delta t) = 1$

Unforgeability: Unforgeability asks that no adversary can forge a signature to pass verification without the corresponding private key. Our proposed LRRS is unforgeable against chosen-message attack (UF-CMA) if for any efficient adversary $\mathscr{A}$, the probability in the following $\mathsf{UF\text{-}CMA}_{\mathscr{A}}^{\lambda}$ experiment is negligible $\Pr[\mathsf{UF\text{-}CMA}_{\mathscr{A}}^{\lambda} = 1] \leqslant \nu(\lambda)$ where ν is a negligible function.

Experiment: $\mathsf{UF\text{-}CMA}_{\mathscr{A}}^{\lambda}$.

$\mathsf{param}_{\mathsf{LRRS}}^{\mathsf{ch}} \leftarrow \mathsf{LRRS.Setup}(\lambda)$

For L, n and t, where $\forall 1 \leqslant i \leqslant n, (sk_i, pk_i) \leftarrow$

$\mathsf{LRRS.UKeyGen}\ (\mathsf{param}_{\mathsf{LRRS}}^{\mathsf{ch}})$

$(tk_{ch}, hk_{ch}) \leftarrow \mathsf{LRRS.UKeyGen}(param_{\mathsf{LRRS}}^{\mathsf{ch}})$

$(m^*, \sigma_L^t(m^*)) \leftarrow \mathscr{A}^{\mathsf{Sign}(\cdots),\mathsf{Redact}(\cdots),\mathsf{Update}(\cdots)}(L, t)$

where Sign, Redact and Update are signing, redacting and update oracles which will return $\perp$ if queried on invalid inputs and response for at most q_s distinct queries in total

Return 1 if $\mathsf{LRRS.Verify}(L^*, m^*, \sigma_{L^*}^t(m^*), \bar{t}) = 1 \Lambda (L^* \subseteq L)$

$\Lambda \sigma_{L^*}^t(m^*) \neq \sigma_L^t(m^i)$ where $i \in [1, q_s]$

and $\bar{t}$ is the current time

else, return 0.

Signer-Ambiguity: Signer-Ambiguity asks that no adversary can link a signature to the identity of signer with non-negligible advantage. Our proposed LRRS is signer-ambiguous if the probability of adversary $\mathscr{A}$ in winning the following experiment is exactly $\frac{1}{|L|}$, i.e. $\Pr|\mathsf{SA}_{\mathscr{A}}^{\lambda}| = \frac{1}{|L|}$.

Experiment: $\mathsf{SA}_{\mathscr{A}}^{\lambda}$.

$\forall \mathsf{param}_{\mathsf{LRRS}} \leftarrow \mathsf{LRRS.Setup}(\lambda)$

$\forall (tk, hk) \leftarrow \mathsf{LRRS.HKeyGen}(\mathsf{param}_{\mathsf{LRRS}})$

Given $(L^*, n^*, \sigma_L^t(m^*))$ where $L^* = \{pk_1, \cdots, pk_n^*\}$

and $\forall 1 \leqslant i \leqslant n^*, (sk_i^*, pk_i^*) \leftarrow \mathsf{LRRS.UKeyGen}\ (\mathsf{param}_{\mathsf{LRRS}})$

$j \xleftarrow{R} \{1, n\}$

$j^* \leftarrow \mathscr{A}_1^{\mathsf{Sign}(\cdots),\mathsf{Redact}(\cdots),\mathsf{Update}(\cdots)}(L^*)$ where oracles Sign, Redact and Update are as previously defined

Return 1 if $(j = j^*) \Lambda (\forall i, j \in [1, n^*],\ \mathsf{LRRS.Link}() = 0)$

Λ (pk_j^* is not on the query list of oracles Sign, Redact or Update)).

else, return 0

Linkability: To prevent double-spending in blockchain, it is needed to detect signatures generated by the same signer even if the user's identity is hidden [5]. For two given LRRS signatures in the same ring under the same set of public keys L, anyone can run LRRS.Link to detect linkability.

Deniability: For any forged LRRS signature, the original signer can deny it by revealing the originally signed signature by running LRRS.Deny.

Signer-accountability: Signer-accountability asks that no malicious signer can forge a signature on a previously signed one at some time without trapdoor key, and successfully pass the LRRS.Deny.

4. The proposed TUCH and LRRS

In this section, we show the construction of TUCH in the GDH group and construction of LRRS. We also give security analysis by following previously defined models.

4.1. Construction of TUCH

TUCH.Setup$(\lambda) \rightarrow (\mathsf{param}_{\mathsf{TUCH}})$: On input a security parameter λ, select a GDH group G generated by g with order q. Set $H_1 : \{0,1\}^* \rightarrow G$ and $H_2 : \{0,1\}^* \rightarrow Z_q$. Define $\Delta t \in G$ as a global time interval for updating chameleon randomness. Output system parameter $\mathsf{param}_{\mathsf{TUCH}} = \{G, q, g, H_1, H_2, \Delta t\}$.

TUCH.KeyGen$(\mathsf{param}_{\mathsf{TUCH}}) \rightarrow (tk, hk)$: On input system parameters $\mathsf{param}_{\mathsf{TUCH}}$, randomly select an integer $x \xleftarrow{R} Z_q^*$ as trapdoor key and set $hk = y = g^x$ as hash key. Output trapdoor key and hash key (tk, hk).

TUCH.Hash$(hk, \mathsf{CID}, m, t, \alpha) \rightarrow (\hbar, r)$: On input a hash key $hk = y$, a customized identity $\mathsf{CID} \in \{0,1\}^*$, a message $m \in \{0,1\}^*$, a current time $t \in Z_q$, and a random number $\alpha \xleftarrow{R} Z_q^*$, compute $h = H_1(\mathsf{CID})$. It then sets chameleon randomness $r = (g^{\alpha t}, y^{\alpha t})$, and computes chameleon hash as $\hbar = g^{\alpha t} h^{H_2(m)t}$. Output $(\hbar, r)$.

TUCH.Verify$(hk, \mathsf{CID}, (m, \hbar, r), t) \rightarrow (\perp,\ 0 \text{ or } 1)$: On input a hash key $hk = x$, a customized identity CID, a current time $t \in Z_q$, and a tuple $(m, \hbar, r)$ which consists of a message m, a chameleon hash $\hbar$ and a chameleon randomness r. First, check whether $< g, g^{\alpha t}, y, y^{\alpha t} >$ is a valid Diffie-Hellman tuple (i.e. in the form of $< g, g^{\alpha t}, g^x, g^{x\alpha t} >$). If not, output $\perp$. Otherwise, compute $h = H_1(\mathsf{CID})$. If $\hbar = g^{\alpha t} h^{H_2(m)t}$ holds, output 1; otherwise, output 0.

TUCH.Forge$(tk, \mathsf{CID}, m', (m, \hbar, r), t) \rightarrow (r' \text{ or } \perp)$: On input a trapdoor key $tk = x$, a customized identity CID, a new message m', a tuple $(m, \hbar, r)$ which includes message m, chameleon hash $\hbar$ and chameleon randomness r, first run TUCH.Verify $(hk, \mathsf{CID}, (m, \hbar, r), t)$ to check the validity. If the output is 1, returns $\perp$ and terminates; otherwise, compute $h = H_1(\mathsf{CID})$ and new chameleon randomness as $r' = (g^{\alpha' t}, y^{\alpha' t}) = (g^{\alpha t} h^{((H_2(m) - H_2(m'))t}$, $y^{\alpha t} h^{x((H_2(m) - H_2(m'))t})$. Then, check whether $< g, g^{\alpha' t}, y, y^{\alpha' t} >$ is a valid Diffie-Hellman tuple (i.e. in the form of $< g, g^{\alpha' t}, g^x, g^{x\alpha' t} >$). If invalid, output $\perp$; otherwise, check whether (1) holds

$$g^{\alpha t} h^{H_2(m)t} \stackrel{?}{=} g^{\alpha' t} h^{H_2(m')t}. \tag{1}$$

If holds it, output r'; otherwise, output $\perp$.

TUCH.Update$(tk, \mathsf{CID}, (m, \hbar, r, t), \Delta t) \rightarrow (r'' \text{ or } \perp)$: On input a trapdoor key $tk = x$, a customized identity CID, a tuple $(m, \hbar, r, t)$, and a time interval Δt, first run TUCH.Verify$(hk, \mathsf{CID}, (m, \hbar, r), t)$ to check the validity. If the output is 1, return $\perp$ and terminate; otherwise, compute $h = H_1(\mathsf{CID})$ and update chameleon randomness as $r'' = (g^{\alpha''(t+\Delta t)}, y^{\alpha''(t+\Delta t)}) = (g^{\alpha t} h^{H_2(m)(-\Delta t)}, y^{\alpha t} h^{xH_2(m)(-\Delta t)})$. Then, check whether $< g, g^{\alpha''(t+\Delta t)}, y, y^{\alpha''(t+\Delta t)} >$ is a valid Diffie-Hellman tuple (i.e. in the form of $< g, g^{\alpha''(t+\Delta t)}, g^x, g^{x\alpha''(t+\Delta t)} >$). If it does not hold, output $\perp$; otherwise, check whether (2) holds

$$g^{\alpha t} h^{H_2(m)t} \stackrel{?}{=} g^{\alpha''(t+\Delta t)} h^{H_2(m)(t+\Delta t)}. \tag{2}$$

If it holds, output r''; otherwise, output $\perp$.

4.2. Security Analysis of TUCH

Correctness: We show correctness of our TUCH based on 1 and 2 as follows.

$$g^{\alpha' t} h^{H_2(m')t} = g^{\alpha t} h^{((H_2(m) - H_2(m'))t} h^{H_2(m')t} = g^{\alpha t} h^{H_2(m)t} h^{[H_2(m') - H_2(m')]t} = g^{\alpha t} h^{H_2(m)t}. \tag{1}$$

$$g^{\alpha''(t+\Delta t)} h^{H_2(m)(t+\Delta t)} = g^{\alpha t} h^{H_2(m)(-\Delta t)} h^{H_2(m)(t+\Delta t)} = g^{\alpha t} h^{H_2(m)t} h^{H_2(m)(\Delta t - \Delta t)} = g^{\alpha t} h^{H_2(m)t}. \tag{2}$$

Indistinguishability: For sufficiently large security parameter λ, any $\mathsf{param}_{\mathsf{TUCH}} \leftarrow$ TUCH.Setup, $(tk, hk) \leftarrow$ TUCH.KeyGen$(\mathsf{param}_{\mathsf{TUCH}})$, all customized identities $\mathsf{CID} \in \{0,1\}$ and all $m, m' \in \{0,1\}$, denote $\mathscr{D}_{TUCH.Forge}$ and $\mathscr{D}_{TUCH.Hash}$ as distribution domains for tuple $(\hbar, r)$ where $\hbar$ and r are generated by TUCH.Hash and TUCH.Forge, respectively. Denote t as the current time and $\mathscr{R}$ as the output domains of chameleon randomness $r = (g^\alpha, y^\alpha)$, we compare the probability distributions of $\mathscr{D}_{TUCH.Hash}$ and $\mathscr{D}_{TUCH.Forge}$ as follows:

$$\mathscr{D}_{TUCH.Hash} =$$

$$\{(r, h) | r \in \mathscr{R},\ m \in \{0,1\}, \hbar \leftarrow \mathsf{TUCH.Hash}(\mathsf{param}_{\mathsf{TUCH}}, \mathsf{CID}, m, t, \alpha)\}$$

$$\mathscr{D}_{\mathsf{TUCH.Forge}} = \{(r', h) | r' \in \mathscr{R},\ m, m' \in \{0,1\},$$

$$\hbar \leftarrow \mathsf{TUCH.Hash}(\mathsf{param}_{\mathsf{TUCH}}, \mathsf{CID}, m, t, \alpha),$$

$$r' \leftarrow \mathsf{TUCH.Forge}(tk, \mathsf{CID}, m', (m, \hbar, t), t)\}$$

For each chameleon hash $\hbar$, a customized identity CID and message m, there is always one-to-one correspondence $r = (\hbar \cdot h^{-H_2(m)}, \hbar \cdot h^{-H_2(m)x})$ where $h = H_1(\mathsf{CID})$ and $\hbar = g^{\alpha t} h^{(H_2(m))t}$. The same goes with $(\hbar, r')$. Therefore, the probability distributions of $\mathscr{D}_{TUCH.Hash}$ and $\mathscr{D}_{TUCH.Forge}$ are indistinguishable.

Collision-resistance: Suppose $\mathscr{A}$ is a PPT adversary against the collision-resistance of our TUCH scheme. We briefly show how to construct an algorithm $\mathscr{B}$ that uses $\mathscr{A}$ to solve CDHP with non-negligible probability.

On given a CDHP instance (g, g^a, g^b), $\mathscr{B}$ interacts with $\mathscr{A}$ to output g^{ab} as follows: At setup stage, $\mathscr{B}$ randomly chooses $x \in Z_q^*$ as trapdoor key and computes $hk = y = g^x$ as hash key. Set $h = g^b$ for an unknown $b \leftarrow Z_p$. $\mathscr{B}$ runs TUCH.Setup to generate $\mathsf{param}_{\mathsf{TUCH}}$. $\mathscr{B}$ sends $\mathsf{param}_{\mathsf{TUCH}}$ and hk to $\mathscr{A}$, and controls random oracles $\mathcal{O}_{H_1}, \mathcal{O}_{H_2}$, Forge, and Update queries.

During query stage, for each distinct query, $\mathscr{B}$ responds differently and records query histories in a list. Thus, for identical query, $\mathscr{B}$ always return the same answer. During output stage, for an output which satisfies a collision $(\mathsf{CID}^*, m, r, (m', r'))$ where $r = (g^a, y^a)$ and $r = (g^{a'}, y^{a'})$, $\mathscr{B}$ computes $(y^{a'}/y^a)^{[H_2(m)-H_2(m')]^{-1}}$ as a solution to the CDHP instance. Since, $(y^{a'}/y^a)^{[H_2(m)-H_1(m')]^{-1}} = h^x$ satisfies CDHP instance (g, g^a, h, h^x), we can reduce the collision-resistance of our TUCH to the CDHP.

4.3. Construction of LRRS scheme

LRRS.Setup$(\lambda) \rightarrow (\mathsf{param}_{\mathsf{TUCH}})$: On input a security parameter λ, select a group G generated by g with order q. Set $H_1 : \{0,1\}^* \rightarrow G$ and $H_2 : \{0,1\}^* \rightarrow Z_q$. Set $\Delta t \in Z_q$ as a global time interval for update. Note that $\Delta t > 0$. Output system parameter $\mathsf{param}_{\mathsf{LRRS}} = \{G, q, g, H_1, H_2, \Delta t\}$.

LRRS.UKeyGen$(\mathsf{param}_{\mathsf{LRRS}}) \rightarrow (sk_i, pk_i)$: On input system parameters $\mathsf{param}_{\mathsf{LRRS}}$, first select a random number $x_i \xleftarrow{R} Z_q$ as the user's private key and generate $y_i = g^{x_i}$ as his public key. Output user's private and public key pair (sk_i, pk_i).

LRRS.HKeyGen$(\mathsf{param}_{\mathsf{LRRS}}) \rightarrow (hk, tk)$: On input system parameters $\mathsf{param}_{\mathsf{LRRS}}$, choose a random integer $tk = x \xleftarrow{R} Z_q^*$ as trapdoor key and compute $hk = y = g^x$ as hash key. Output trapdoor key and hash key (tk, hk).

LRRS.Sign$(hk, L, m, x_\pi, t) \rightarrow (\sigma_L^t(m))$: On input a hash key $hk = y = g^x$ where x is denoted as trapdoor key, a list of n public keys $L = \{y_1, \cdots, y_n\}$, signer's private key x_π where $1 \leqslant \pi \leqslant n$ and a current time $t \in Z_q$, compute a LRRS signature at time t as follows:

1. Set $\mathsf{CID} = L$ as a customized identity. Compute $h = H_1(\mathsf{CID})$ and $\tilde{y} = h^{x_\pi}$.
2. Pick two random numbers $u, v \xleftarrow{R} Z_q$ and compute $c_{\pi+1} = H_1(L, \tilde{y}, g^{ut}, h^v)$.
3. For each $i = \pi+1, \cdots, n, 1, \cdots, \pi-1$, pick two random numbers $\alpha_i, \beta_i \xleftarrow{R} Z_q^*$, and run TUCH.Hash$(hk, \mathsf{CID}, m, t, \alpha_i)$ to output chameleon hash $\hbar_i$ and chameleon randomness $r_i = (g^{\alpha t}, y^{\alpha t})$. Then, compute

$$c_{i+1} = H_2(L, \tilde{y}, \hbar_i \cdot y_i^{c_i}, h^{\beta_i} \cdot \tilde{y}^{c_i}).$$

4. Then, compute $\alpha_\pi = u - x_\pi c_\pi t^{-1} \bmod q$, $\beta_\pi = v - x_\pi c_\pi \bmod q$.

Output $\sigma_L^t(m) = (c_1, r_1, \cdots, r_n, \beta_1, \cdots, \beta_n, \tilde{y})$.

LRRS.Verify$(L, m, \sigma_L^t(m), \bar{t}) \rightarrow (0 \text{ or } 1)$: On input a list of n public keys $L = \{y_1, \cdots, y_n\}$, a message $m \in \{0,1\}^*$, LRRS signature at time t as $\sigma_L^t(m) = (c_1, r_1, \cdots, r_n, \beta_1, \cdots, \beta_n, \tilde{y})$ and a current time $\bar{t}$, verify as follows:

1. Set $\mathsf{CID} = L$. For $i = 1, \cdots, n$, compute $\hbar_i = g^{\alpha \bar{t}} h^{H_2(m)\bar{t}}$ where $r_i = (g^{\alpha t}, y^{\alpha t})$ and t is unknown to verifier. Compute $z_i' = \hbar_i \cdot y_i^{c_i}$, $z_i'' = h^{\beta_i} \cdot \tilde{y}^{c_i}$. Then, compute $c_{i+1} = H_2(L, \tilde{y}, z_i', z_i'')$ for $i \neq n$.
2. Check whether $c_1 \stackrel{?}{=} H_2(L, \tilde{y}, z_n', z_n'')$. If it holds, output 1; otherwise, output 0 which signifies the signature is either invalid or out of date (i.e. $t \neq \bar{t}$).

LRRS.Redact$(tk, L, m', \sigma_L^t(m), t) \rightarrow (\perp \text{ or } \sigma_L(m'))$: On input a trapdoor key $tk = x$, a list of n public keys $L = \{y_1, \cdots, y_n\}$, a new message $m' \in \{0,1\}^*$, LRRS signature at time t as $\sigma_L^t(m) = (c_1, r_1, \cdots, r_n, \beta_1, \cdots, \beta_n, \tilde{y})$ and a time point t, first run LRRS.Verify$(L, m, \sigma_L^t(m), t)$ to check the validity. If the output is 0, return $\perp$ and terminate; otherwise, proceed as follows. Set $\mathsf{CID} = L$. For $1 \leqslant i \leqslant n$, compute each new randomness $r_i' = (g^{\alpha_i' t}, y^{\alpha_i' t}) = (g^{\alpha_i' t} h^{[(H_2(m)-H_2(m'))]t}, y^{\alpha_i' t} h^{x[(H_2(m)-H_2(m'))]t})$. Output a redacted LRRS signature $\sigma_L^t(m') = (c_1, r_1', \cdots, r_n', \beta_1, \cdots, \beta_n, \tilde{y})$ at time point t.

LRRS.Update$(tk, L, m, \sigma_L^t(m), t, \Delta t) \rightarrow (\perp \text{ or } \sigma_L^{t+\Delta t}(m))$: On input a trapdoor key $tk = x$, a list of n public keys $L = \{y_1, \cdots, y_n\}$, a message $m \in \{0,1\}^*$, LRRS signature $\sigma_L^t(m) = (c_1, r_1, \cdots, r_n, \beta_1, \cdots, \beta_n, \tilde{y})$ at time t, a time point t and a global time interval $\Delta t > 0$, first run LRRS.Verify$(L, m, \sigma_L(m), t)$ to check the validity. If the output is either $\perp$ or 0, return $\perp$ and terminate; otherwise, proceed the following update: Set $\mathsf{CID} = L$. For $1 \leqslant i \leqslant n$, compute $r_i'' = (g_i^{\alpha_i''(t+\Delta t)}, y^{\alpha_i''(t+\Delta t)}) = (g_i^{\alpha t} h^{H_2(m)(-\Delta t)}, y_i^{\alpha t} h^{xH_2(m)(-\Delta t)})$ to derive each updated randomness r_i''. Output an updated LRRS signature $\sigma_L^{t+\Delta t}(m) = (c_1, r_1'', \cdots, r_n'', \beta_1, \cdots, \beta_n, \tilde{y})$ at time point $t + \Delta t$.

LRRS.Link$(L, (pk_{d_0}, m_{d_0}, \sigma_L^t(m_{d_0})), (pk_{d_1}, m_{d_1}, \sigma_L^t(m_{d_1})), \bar{t}) \rightarrow (\perp, 0 \text{ or } 1)$: On input a list of n public keys $L = \{y_1, \cdots, y_n\}$, a tuple $(pk_{d_i}, m_{d_i}, \sigma_L^t(m_{d_i}))$ where $d_i \in L$ and $t = (0,1)$, and a current time $\bar{t}$, check:

1. Run LRRS.Verify$(L, m_{d_i}, \sigma_L^t(m_{d_i}))$ for $i = (0, 1)$, if the output is 1, go to next step; otherwise, return $\perp$,
2. Check whether $\tilde{y}_{d_0} = \tilde{y}_{d_1}$, if it holds, output 1; otherwise, output 0.

LRRS.Deny$(L, m^*, \sigma_t^L(m^*), m, \sigma_t^L(m), \bar{t}) \rightarrow (\perp,$ 0 or 1): On input a list of n public keys $L = \{y_1, \cdots, y_n\}$, dispute message m^* and signature $\sigma_t^L(m^*)$, original message m and signature $\sigma_t^L(m)$, a current time $\bar{t}$, first run LRRS.Verify on $\sigma_t^L(m^*)$ and $\sigma_t^L(m)$ to check the validity. If it fails, output $\perp$ and terminate; otherwise, check whether if any collisions exist: TUCH.Hash$(\mathsf{CID}, m^*, r_i^*) =$ TUCH.Hash(CID, m, r_i) for any $i \in [1, n]$ where $\mathsf{CID} = L$ is denoted as a customized identity. If not hold, output 0; otherwise, output 1.

4.4. Security analysis of LRRS

Correctness: The correctness of our LRRS is clear from inspection.

Unforgeability: Our proposed LRRS is unforgeable against chosen-message attack if the DLP is hard in the random oracle. Briefly, suppose a probabilistic polynomial time (PPT) adversary $\mathscr{A}$ against the unforgeability of our LRRS makes at most q_s queries to Sign, Redact and Update oracles to forge a valid LRRS signature with non-negligible probability. Then, we can construct a simulator $\mathscr{M}$ which uses $\mathscr{A}$ to solve DLP with one of public keys in L. Denote the successful forgery generated by $\mathscr{A}$ as below:

$$\sigma^* = (c_1^*, r_1^*, \cdots, r_n^*, \beta_1^*, \beta_n^*, \tilde{y}^*)$$

where the output signature σ^* of $\mathscr{A}$ against our unforgeability experiment satisfies following equations:

$$c_{i+1}^* = H_2(L, \tilde{y}, \hbar \cdot y_i^* \cdot c_i^*, h^{\beta_i^*} \cdot \tilde{y}^{c_i^*}),$$

$$c_1^* = H_2(L, \tilde{y}, \hbar \cdot y_n^{*c_n^*}, h^{\beta_n^*} \cdot \tilde{y}^{c_n^*}).$$

On input of the DLP instance (g, g^x), $\mathscr{M}$ sets $y_{x_\pi} = g^x$ where $x_\pi \in L$, it aims to output x. $\mathscr{M}$ simulates inputs for $\mathscr{A}$ and processes outputs from $\mathscr{A}$. It records the queries history of $\mathscr{A}$ in a list and returns identity response upon receiving the same query. Some outputs of $\mathscr{A}$ are valid forgeries of LRRS and are used to solve DLP with non-negligible probability (Refer to [34] for details).

Signer-Ambiguity: Our proposed LRRS is Signer-Ambiguous if the DDHP is hard in the random oracle. Briefly, suppose $\mathscr{A}$ is a PPT adversary against the signer-ambiguity of our LRRS, then we can construct a PPT simulator $\mathscr{S}$ to solve DDHP by using $\mathscr{A}$. Concretely, on given a message m, a list of n public keys L, a set of private keys $SKT = \{sk_i, \cdots, sk_t\}$ where $0 \leqslant t \leqslant n-1$, a signature σ_L^t signed with private key sk_π at current time t on message m, $\mathscr{S}$ can be constructed to solve DDHP for some polynomial $Q(k)$ with probability:

$$Pr[\mathscr{A}(m, L, SKT, \sigma)] > \frac{1}{n-t} + \frac{1}{Q(k)}.$$

Refer to [34] for details.

Linkability: The linkability of our LRRS can be reduced to CDHP. For a fixed L, the same signer with private key x_π will always produce same $\tilde{y} = H_1(L)^{x_\pi}$. Following this, any user can run LRRS.Link to detect linkability generated from same signer. The reduction is easy from inspection. Refer to [34] for details.

Deniability: When disputing a forged LRRS signature, algorithm.

LRRS.Deny allows the original signer to deny the disputed signature by revealing the original signature. Concretely, suppose the dispute message and signature pair are: $(m^*, \sigma_t^L(m^*))$ where the signature is forged by the holder of trapdoor key tk (i.e., miner), the original signer can deny it by revealing $(m, \sigma_t^L(m))$. Since collisions are found between dispute pair $(m^*, \sigma_t^L(m^*))$ and original pair $(m, \sigma_t^L(m))$, and only the trapdoor key tk holder can forge it by running TUCH.Forge. This suggests that he must have made this forgery during a past time. Therefore, we can reduce our deniability to the collision-resistance of our TUCH as proved in Section 4.2.

Signer-accountability: Conversely, we suppose a malicious signer who can forge a previously signed signature and pass the LRRS.Deny without trapdoor key tk. Therefore, he can find the collisions to redact the signed signature in a past time, this breaks the collision-resistance of TUCH. Therefore, we can reduce our signer-accountability to the collision-resistance of our TUCH as proved in Section 4.2.

5. Instantiation

In this section, we briefly instantiate how to build scalable and redactable blockchain (SRB) with proposed TUCH and LRRS.

5.1. Basic idea

Based on the idea introduced by Ateniese et al. [7], redactable blockchain is powered by replacing the system hash function (i.e. SHA-256) and digital signature scheme (i.e. ECDSA) with chameleon hash and malleable signature [14]. In this work, our proposed TUCH and LRRS serve as hash function and signing scheme, respectively. For clarity, we adapt basic bitcoin transaction slightly for redaction [36] and show an example in Fig. 2.

5.2. Key generation

To initiate our SRB, key generation is required. Instead of generating and distributing trapdoor keys among several validators [9], we allow the corresponding miner to dictate block redaction with his own trapdoor key *tk*. Simply, we run TUCH.KeyGen for assigning *tk* and *hk* to the miner. Then, the redaction power of a specific block is controlled by the miner who appended the block to a chain. The implementation is easy: the miner will additionally record his hash key *hk* on the block (as shown in Fig. 2) so that he can later redact it when summoned. How to reach an agreement on the redacted block is another challenging issue, which we will investigate it in the future.

Algorithm 1. Block Redaction based on TUCH and LRRS

Input:
An original block $\mathscr{B} = ((L, \hbar_{prev}, r, m), tran)$, a new transaction set $tran'$, a trapdoor key tk and a current time t.
Here, denote $tran = \{m_i, \sigma_L^t(m_i)\}_{1 \leqslant i \leqslant n}$ as a set of transactions, L as a list of n public keys, $\hbar_{prev}$ as hash of previous block which satisfies $\hbar = \mathsf{TUCH.Hash}(\mathsf{hk}, \mathsf{CID}, \mathsf{m}, \mathsf{t}, \mathsf{r})$ block, m as a nonce [7] and $\{\sigma_L^t(m)_i\}$ as a LRRS signature.
Output: The redacted block $\mathscr{B}' = ((\mathsf{CID}, \hbar_{prev}, r', m'), tran')$
1: set $\mathsf{CID} = L$
2: run $r' \leftarrow \mathsf{TUCH.Forge}(tk, \mathsf{CID}, m', \hbar_{prev}, r, m, t)$;
3: **for** each $i \in [1, n]$
4: run $tran' \leftarrow \mathsf{LRRS.Redact}(tk, L, tran'_i, \{\sigma_L^t(m)_i\}, t)$;
5: **end for**
return $\mathscr{B}'$

5.3. Instantiating block redaction

We instantiate a block redaction by the pseudocode in Algorithm 1. As instantiated, miner runs TUCH.Forge and LRRS.Redact to redact committed hash and signature while keeping block hash consistency. After that, major nodes can keep

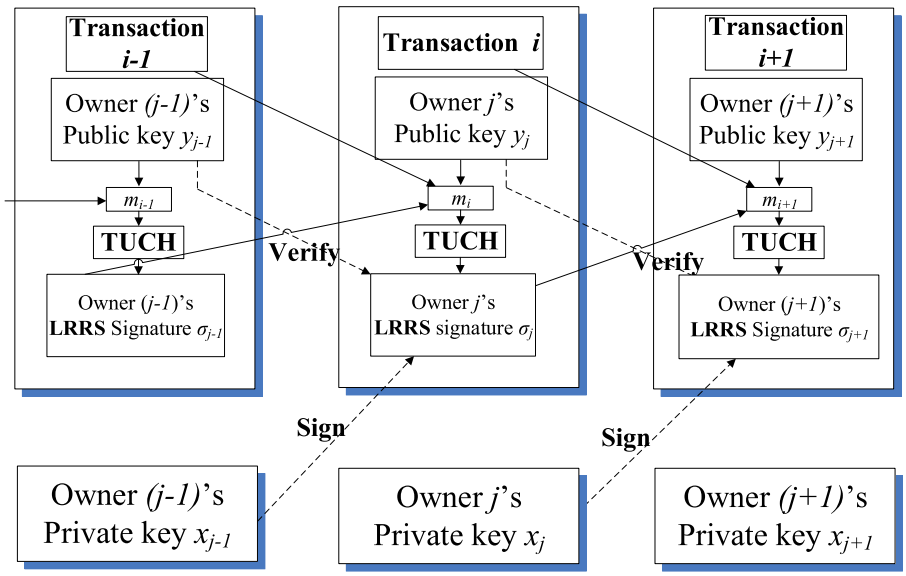

Fig. 2. Adapting bitcoin transaction for redaction.

Table 4
Complexity of hashing schemes.

Work	Algorithms			
	Hash	Verification	Forge	Update
TUCH	$3T_e + 3T_m$	$3T_e + 3T_m$	$2T_e + 4T_m$	$2T_e + 4T_m$
ACH [31]	$2T_m + 1T_p$	$1T_e + 2$ $T_m + 1T_p$		n/a
CHKE [2]	$4T_e + 2T_m$	$4T_e + 2T_m$	$2T_e$+ $2T_m + 1T_i$	n/a
IDCH [18]	$3T_e + 1T_m$	$3T_e + 1T_m$	$2T_e + 2T_m$	n/a

Denote T_m as group multiplication; T_e as group exponentiation; T_i as group inversion; T_p as bilinear pairing operation; n/a as not applicable.

Table 5
Complexity of signature schemes.

Schemes	Algorithms			
	Sign	Verify	Redact	Update
Our LRRS	$(5n-1)T_e + (4n-1)T_m$	$(2n+2)T_e + (3n+1)T_m$	$(4n+2)T_e + (7n+1)T_m$	$2nT_e + 4nT_m$
	$3T_e + 1T_m + 1T_s$	$2T_e + 1T_m + 1T_v$	$2T_e + 1T_m + 1T_i$	n/a
[3]	$4T_e + 2T_m + 1T_s$	$2T_e + 1T_m + 1T_v$	$2T_e + 3T_m + 1T_i$	n/a
[17]	$3T_e + 2T_m + 1T_s$	$2T_e + 2T_m + 1T_v$	$2T_e + 3T_m + 1T_i$	n/a

Denote n as number of ring members; T_m as group multiplication; T_e as group exponentiation; T_i as group inversion; T_p as bilinear pairing operation; n/a as not applicable; T_s as signing operation of BLS signature [10] signature; T_s as verification operation of BLS signature [10].

mining on the redacted chain. Additionally, the miner needs to update redacted chain (by running TUCH.Update and LRRS.Update periodically); otherwise, other miners will stop following the redacted results after Δt time (Δt is a configurable timeout) and switch to the original chain instead. Therefore, redaction is enabled while preventing abuse of the redaction power. Since the update process is similar to the redaction procedure, details are omitted here.

6. Performance evaluation

In this section, we give both complexity and experimental analysis on our proposed TUCH and LRRS schemes. To test algebraic and cryptographic algorithms, we implement PBC library for simulations. Our experimental platform is a laptop equipped with Intel i5-3210 M CPU and dual-core processors of 2.3 GHz with 4 GB RAM. The operating system is Windows 7 SP1. We use OpenSSL as a means of communication.

6.1. Complexity analysis

We compare the complexity of our proposed TUCH and LRRS with similar schemes in Tables 4 and 5. As it is shown in Table 4, our TUCH is as efficient as other works in major operations, including hashing, verification and forging. As it is

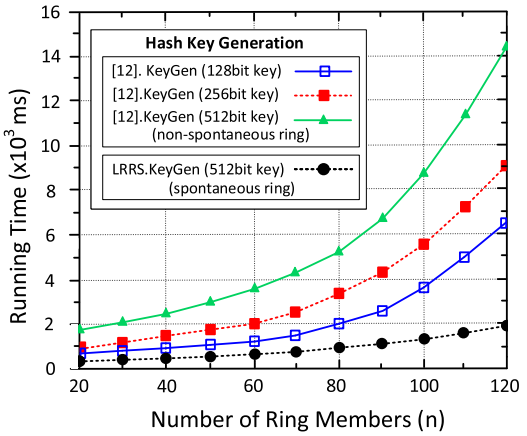

Fig. 3. KeyGen cost.

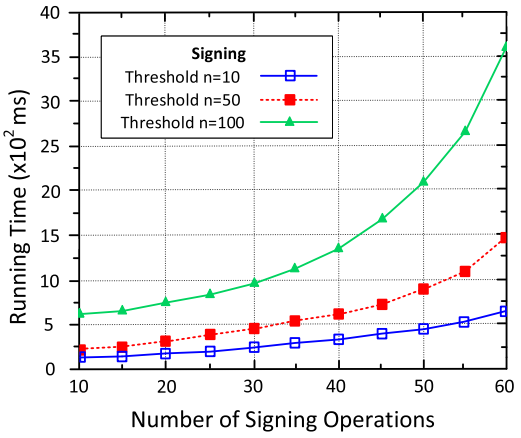

Fig. 4. Signing cost.

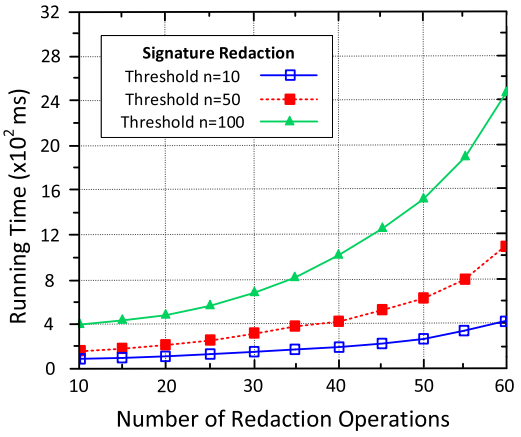

Fig. 5. Redaction cost.

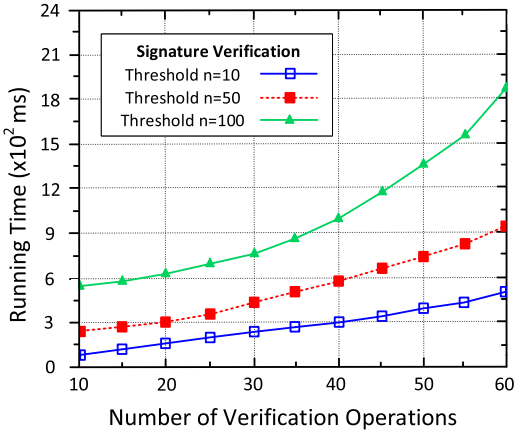

Fig. 6. Verification cost.

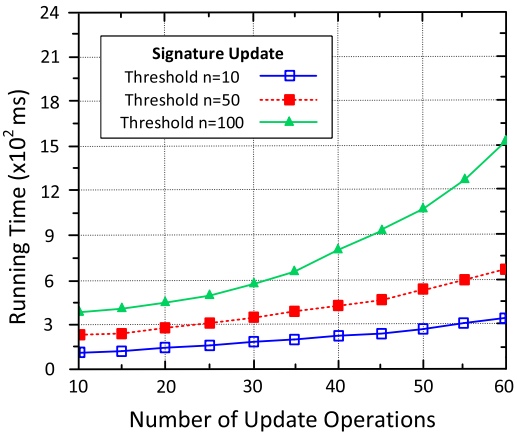

Fig. 7. Update cost.

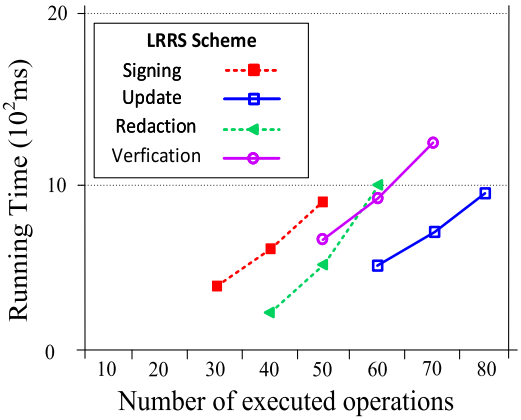

Fig. 8. Overview of cost.

shown in Table 5, our LRRS is dominated by the size of the ring, which is chosen for signing. Although our LRRS requires linear costs (with n) in generating and processing signature, these operations yield anonymity, scalability and security features. Meanwhile, if n is not large, associated cryptographic costs are still acceptably low.

6.2. Experimental analysis

To test our LRRS's performance in key generation, we compare it with our previous work [26] in Fig. 3. We apply different key sizes (128, 256, 512 bits) to our previous work, and apply 512 bit key length to our LRRS to test key generation performance. As it is depicted in Fig. 3, our previous work [26] which is based on spontaneous ring formation yields an obvious linear cost in key generation as it requires the participation of each ring member. Differently, our LRRS yields a much stable cost in key generation. We also list the costs of different dominant operations in LRRS in Figs. 4–7. The results show that each function of our LRRS is dominated by the factor of threshold n (i.e., the number of nodes in the spontaneous ring). For a quantitative view, we give an overview in Fig. 8. As it is shown in Fig. 8, within an approximately 100 ms computing time where we fix threshold n by 50 (i.e., a ring of 50 members), it yields 50 times of signing operations, 60 times of redaction and verification operations, 80 times of an update. Clearly, signing is the most time-consuming operation while the update is the least one in our LRRS, and therefore when n is not surprisingly large, our LRRS is acceptably efficient.

7. Conclusion

In this paper, we propose a scalable and redactable blockchain with our new theoretical tools. Specifically, we propose a time updatable chameleon hash (TUCH) and a linkable-and-redactable ring signature (LRRS) as theoretical building blocks. We instantiate how to build a redactable blockchain with update and anonymity features where the user's privacy is pre-

served, and the misuses of redaction are avoided. Our security analysis shows our scheme satisfies security requirements. Experimental results show that our proposals are valid and efficient to be implemented. Meanwhile, it also illustrates that the cost of redaction is factored by the size of the ring selected by the signer. In the future, we will investigate how to boost efficiency for the redaction of blockchain and crypto-currencies. This helps us build both reliable and efficient trust layer for IoT devices.

CRediT authorship contribution statement

Ke Huang: Conceptualization, Methodology, Software, Writing - original draft, Visualization. **Xiaosong Zhang:** Validation, Formal analysis, Resources, Data curation, Supervision, Project administration, Funding acquisition. **Yi Mu:** Supervision, Writing - review & editing. **Fatemeh Rezaeibagha:** Writing - review & editing. **Xiaojiang Du:** Funding acquisition.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported in part by the National Key R&D Program of China under Grant No. 2017YFB0802300 and Grant No. 2018YFB08040505; National Natural Science Foundation of China under Grants 61872087, 61602092 and U19A2066; Research Initiation Fund (Central Universities) under Grant No. Y030202059018061 and Y030202059018060.

References

[1] Au. Man Ho, Sherman S.M. Chow, Willy Susilo, Patrick P. Tsang, Short linkable ring signatures revisited, in: European Public Key Infrastructure Workshop, Springer, 2006, pp. 101–115.
[2] Giuseppe Ateniese, Breno de Medeiros, Identity-based chameleon hash and applications, in: International Conference on Financial Cryptography, Springer, 2004, pp. 164–180.
[3] Giuseppe Ateniese, Breno de Medeiros, On the key exposure problem in chameleon hashes, in: International Conference on Security in Communication Networks, Springer, 2004, pp. 165–179.
[4] Luigi Atzori, Antonio Iera, Giacomo Morabito, The internet of things: a survey, Computer Networks 54 (15) (2010) 2787–2805.
[5] Au. Man Ho, Joseph K Liu, Willy Susilo, Tsz Hon Yuen, Constant-size id-based linkable and revocable-iff-linked ring signature, in: International Conference on Cryptology in India, Springer, 2006, pp. 364–378.
[6] Man Ho Au, Joseph K. Liu, Willy Susilo, Tsz Hon Yuen, Secure id-based linkable and revocable-iff-linked ring signature with constant-size construction, Theoretical Computer Science 469 (2013) 1–14.
[7] Giuseppe Ateniese, Bernardo Magri, Daniele Venturi, Ewerton Andrade, Redactable blockchain–or–rewriting history in bitcoin and friends, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017, pp. 111–126.
[8] Muneeb Ali, Jude Nelson, Ryan Shea, Michael J. Freedman, Blockstack: A global naming and storage system secured by blockchains, in: 2016 {*USENIX*} Annual Technical Conference ({*USENIX*}{*ATC*}16), 2016, pp. 181–194.
[9] Kondapally Ashritha, M. Sindhu, K.V. Lakshmy, Redactable blockchain using enhanced chameleon hash function, in: 2019 5th International Conference on Advanced Computing & Communication Systems (ICACCS), IEEE, 2019, pp. 323–328.
[10] Dan Boneh, Ben Lynn, Hovav Shacham, Short signatures from the weil pairing, in: International Conference on the Theory and Application of Cryptology and Information Security, Springer, 2001, pp. 514–532.
[11] Mihir Bellare, Todor Ristov, A characterization of chameleon hash functions and new, efficient designs, Journal of Cryptology 27 (4) (2014) 799–823.
[12] Emmanuel Bresson, Jacques Stern, Michael Szydlo, Threshold ring signatures and applications to ad-hoc groups, in: Annual International Cryptology Conference, Springer, 2002, pp. 465–480.
[13] Jan Camenisch, David Derler, Stephan Krenn, Henrich C. Pöhls, Kai Samelin, Daniel Slamanig, Chameleon-hashes with ephemeral trapdoors, in: IACR International Workshop on Public Key Cryptography, Springer, 2017, pp. 152–182.
[14] Melissa Chase, Markulf Kohlweiss, Anna Lysyanskaya, Sarah Meiklejohn, Malleable signatures: new definitions and delegatable anonymous credentials, in: 2014 IEEE 27th Computer Security Foundations Symposium, IEEE, 2014, pp. 199–213.
[15] Xiaofeng Chen, Haibo Tian, Fangguo Zhang, Yong Ding, Comments and improvements on key-exposure free chameleon hashing based on factoring, in: International Conference on Information Security and Cryptology, Springer, 2010, pp. 415–426.
[16] David Chaum, Eugène Van Heyst, Group signatures, in: Workshop on the Theory and Application of of Cryptographic Techniques, Springer, 1991, pp. 257–265.
[17] Xiaofeng Chen, Fangguo Zhang, Kwangjo Kim, Chameleon hashing without key exposure, in: International Conference on Information Security, Springer, 2004, pp. 87–98.
[18] Xiaofeng Chen, Fangguo Zhang, Haibo Tian, Baodian Wei, Kwangjo Kim, Key-exposure free chameleon hashing and signatures based on discrete logarithm systems, IACR Cryptology ePrint Archive 2009 (2009) 35.
[19] X. Deng, C.H. Lee, H. Zhu, Deniable authentication protocols, IEE Proceedings-Computers and Digital Techniques 148 (2) (2001) 101–104.
[20] Dominic Deuber, Bernardo Magri, Sri Aravinda Krishnan Thyagarajan. Redactable blockchain in the permissionless setting, arXiv preprint arXiv:1901.03206, 2019.
[21] Ittay Eyal, Adem Efe Gencer, Emin Gün Sirer, Robbert Van Renesse, Bitcoin-ng: A scalable blockchain protocol, in: 13th {USENIX}) Symposium on Networked Systems Design and Implementation (NSDI) 16), 2016, pp. 45–59.
[22] Eiichiro Fujisaki, Koutarou Suzuki, Traceable ring signature, in: International Workshop on Public Key Cryptography, Springer, 2007, pp. 181–200.
[23] Eiichiro Fujisaki, Sub-linear size traceable ring signatures without random oracles, in: Cryptographers Track at the RSA Conference, Springer, 2011, pp. 393–415.
[24] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, Marimuthu Palaniswami, Internet of things (iot): a vision, architectural elements, and future directions, Future Generation Computer Systems 29 (7) (2013) 1645–1660.
[25] Ke Huang, Xiaosong Zhang, Yi Mu, Fatemeh Rezaeibagha, Xiaojiang Du, Nadra Guizani, Achieving intelligent trust-layer for iot via self-redactable blockchain, IEEE Transactions on Industrial Informatics, 2019.

[26] Ke Huang, Xiaosong Zhang, Yi Mu, Xiaofen Wang, Guomin Yang, Xiaojiang Du, Qi Xia, Fatemeh Rezaeibagha, Mohsen Guizani, Building redactable consortium blockchain for industrial internet-of-things, IEEE Transactions on Industrial Informatics, 2019.
[27] Stephan Krenn, Henrich C Pöhls, Kai Samelin, Daniel Slamanig, Chameleon-hashes with dual long-term trapdoors and their applications, in: International Conference on Cryptology in Africa, Springer, 2018, pp. 11–32.
[28] Hugo Mario Krawczyk, Tal D. Rabin, Chameleon hashing and signatures, August 22 2000, US Patent 6,108,783.
[29] Changsu Kim, Wang Tao, Namchul Shin, Ki-Soo Kim, An empirical study of customers perceptions of security and trust in e-payment systems, Electronic Commerce Research and Applications 9 (1) (2010) 84–95.
[30] Joseph K Liu, Au. Man Ho, Willy Susilo, Jianying Zhou, Linkable ring signature with unconditional anonymity, IEEE Transactions on Knowledge and Data Engineering 26 (1) (2013) 157–165.
[31] Junzuo Lai, Xu.hua. Ding, Wu. Yongdong, Accountable trapdoor sanitizable signatures, in: International Conference on Information Security Practice and Experience, Springer, 2013, pp. 117–131.
[32] Iuon-Chang Lin, Tzu-Chun Liao, A survey of blockchain security issues and challenges, IJ Network Security 19 (5) (2017) 653–659.
[33] Joseph K. Liu, Duncan S. Wong, Linkable ring signatures: Security models and new schemes, in: International Conference on Computational Science and Its Applications, Springer, 2005, pp. 614–623.
[34] Joseph K Liu, Victor K Wei, Duncan S Wong, Linkable spontaneous anonymous group signature for ad hoc groups, in: Australasian Conference on Information Security and Privacy, Springer, 2004, pp. 325–335.
[35] Roman Matzutt, Martin Henze, Jan Henrik Ziegeldorf, Jens Hiller, Klaus Wehrle, Thwarting unwanted blockchain content insertion, in: 2018 IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2018, pp. 364–370.
[36] Satoshi Nakamoto, et al., Bitcoin: A peer-to-peer electronic cash system, 2008 .
[37] Ivan Puddu, Alexandra Dmitrienko, Srdjan Capkun, μ chain: How to forget without hard forks. IACR Cryptology ePrint Archive, 2017:106, 2017.
[38] Reversecoin worlds first cryptocurrency with reversible transactions, 2014, https://bitcoinist.com/reversecoin-worlds-first-cryptocurrencyreversible-transactions/.
[39] Shi-Feng Sun, Man Ho Au, Joseph K. Liu, Tsz Hon Yuen, Ringct 2.0: A compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero, in: European Symposium on Research in Computer Security, Springer, 2017, pp. 456–474.
[40] Simon Schwerin. Blockchain and privacy protection in the case of the european general data protection regulation (gdpr): A delphi study, 2018.
[41] Patrick P Tsang, Victor K Wei, Short linkable ring signatures for e-voting, e-cash and attestation, in: International Conference on Information Security Practice and Experience, Springer, 2005, pp. 48–60.
[42] Patrick P Tsang, Victor K Wei, Tony K Chan, Au. Man Ho, Joseph K Liu, Duncan S Wong, Separable linkable threshold ring signatures, in: International Conference on Cryptology in India, Springer, 2004, pp. 384–398.
[43] Marko Vukolić, The quest for scalable blockchain fabric: Proof-of-work vs. bft replication, in: International Workshop on Open Problems in Network Security, Springer, 2015, pp. 112–125.
[44] Jie Xu, Kaiping Xue, Hangyu Tian, Jianan Hong, Peilin Hong, An identity management and authentication scheme based on redactable blockchain for mobile networks, IEEE Transactions on Vehicular Technology PP(99):1–1, 2020.
[45] Tsz Hon Yuen, Joseph K Liu, Man Ho Au, Willy Susilo, Jianying Zhou, Efficient linkable and/or threshold ring signature without random oracles, The Computer Journal, 56(4) (2013) 407–421.